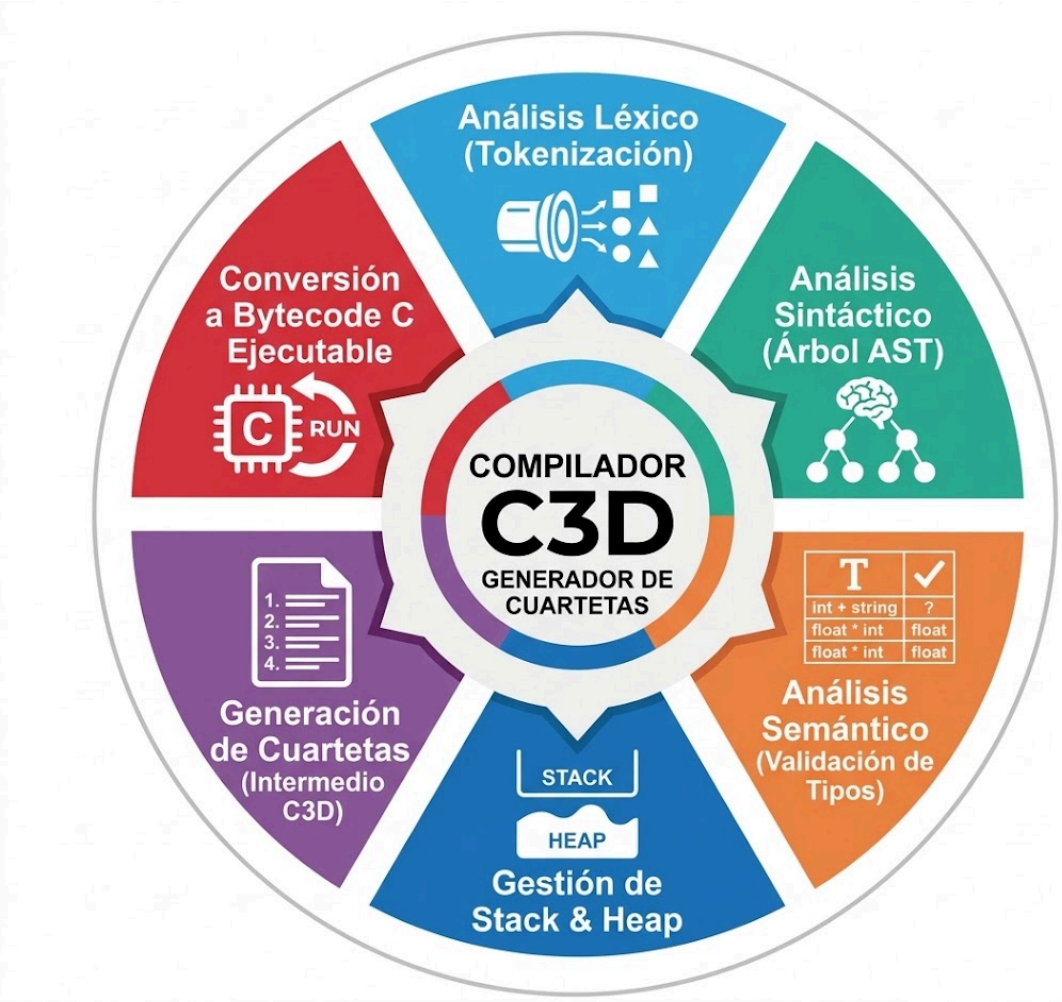
MANUAL TÉCNICO

IDE XS2



Javier Alejandro Mérida Gómez 202230638

27/10/2026

ÍNDICE

DESCRIPCIÓN GENERAL 3
OBJETIVOS 3
- GENERAL 3
- ESPECÍFICOS 3

ESPECIFICACIONES DEL SISTEMA 4
- REQUISITOS DE HARDWARE Y SOFTWARE 4
- TECNOLOGÍAS 4
- REPOSITORIO EN GITHUB 5
- ORGANIZACIÓN DEL PROYECTO 5
- ANÁLISIS DE LOS REQUERIMIENTOS 6
  - PIGLATIN 6
  - Y 10
  - ZETARIANO 15
- ANALIZADOR SEMÁNTICO 21
- CÓDIGO DE TRES DIRECCIONES 23
- TRADUCCIÓN A RISC-V 23
- TABLA DE COLOREADO 25
- INTERFAZ GRÁFICA 26

DESCRIPCIÓN GENERAL

IDE XS2 es un entorno de desarrollo para tres lenguajes de alto nivel: Pig Latin, Y? y Zetariano. Valida cada programa de forma léxica, sintáctica y semántica, lo traduce a código de tres direcciones por medio de cuartetas y, a partir de ellas, genera código ensamblador RISC-V funcional.

Durante la compilación se pueden consultar la tabla de símbolos, la tabla de tipos, la tabla de compatibilidad, las cuartetas, el código de tres direcciones y el código RISC-V generado.

OBJETIVOS

GENERAL

- Traducir programas escritos en tres lenguajes de alto nivel a ensamblador RISC-V funcional, pasando por el código de tres direcciones.

ESPECÍFICOS

- Validar los programas de manera léxica, sintáctica y semántica antes de generar código, incluyendo las reglas de programación orientada a objetos de Zetariano.
- Implementar herencia simple, visibilidad, sobrescritura con @Override, super y despacho dinámico.
- Generar las cuartetas y el código de tres direcciones usando únicamente el stack y el heap, sin funciones auxiliares.
- Traducir las cuartetas a ensamblador RISC-V funcional, con P como frame pointer.

ESPECIFICACIONES DEL SISTEMA

REQUISITOS DE HARDWARE Y SOFTWARE

- Sistemas Operativos Compatibles: Microsoft Windows 10 (o superior) / GNU/Linux (Ubuntu 20.04 LTS o superior).
- Java 21 (JDK) para ejecutar el IDE y Apache Maven 3 para compilarlo.
- RARS 1.6 (RISC-V Assembler and Runtime Simulator) para ejecutar el ensamblador generado.
- Entorno de Desarrollo: Visual Studio Code y Apache NetBeans.

TECNOLOGÍAS

Desarrollo	**Tecnología / Librería**	**Función**
Analizadores (léxico y sintáctico)	ANTLR4 4.13.2	Generar el lexer, el parser y la interfaz Visitor de cada lenguaje a partir de su gramática (pigLatin.g4, y.g4 y Zetariano.g4).
Interfaz gráfica	Java Swing	Interfaz de escritorio: árbol del proyecto, editor con pestañas, tabla de errores, panel de código RISC-V y diálogos del menú Generado.
Backend	Java 21	Tablas de símbolos y tipos, análisis semántico en los visitors, cuartetas, código de tres direcciones y traducción a RISC-V.
Código final	RISC-V + RARS	Ensamblador RV32IMFD generado a partir de las cuartetas, que se ejecuta en el simulador RARS.

REPOSITORIO EN GITHUB

Link del repositorio: https://github.com/JavierMeridaLk/ProyectoDosCompi2SS26

ORGANIZACIÓN DEL PROYECTO

Paquete	Contenido
analizador	Análisis léxico y sintáctico, datos de cada lenguaje, carga del programa y sus imports y orden de los dos recorridos.
tipos	Tipo, TipoPrimitivo, Visibilidad, operadores, literales y posiciones.
visitors	Un paquete por lenguaje con sus dos visitors y TraductorBase, el padre común de los traductores.
tablas	Tabla de símbolos, tabla de tipos, tabla de compatibilidad, ámbitos y tabla hash.
cuartetas	Cuarteta, ListaCuartetas, Valor, Direccion y TraductorC3D.
riscv	TraductorRiscV: traducción de las cuartetas a ensamblador RISC-V.
errores	Manejo y reporte de errores léxicos, sintácticos y semánticos.
archivos	Lectura y escritura de archivos y tabulación del código.
views	Interfaz gráfica: ventana principal, paneles, diálogos, coloreado y componentes.
com.mycompany	Lexers, parsers y visitors generados por ANTLR (carpeta src/main/antlr4).

ANÁLISIS DE LOS REQUERIMIENTOS

El compilador reconoce tres lenguajes. Cada uno tiene su propia gramática de ANTLR genera su lexer, su parser y su clase base Visitor.

Los errores léxicos no detienen el análisis: los símbolos inválidos se envían a un canal oculto y se reportan, los errores sintácticos se reportan con una estrategia de recuperación en español que indica qué se esperaba y, cuando falta un ; , lo marca al final de la instrucción anterior.

PIGLATIN

Analizador sintáctico — gramática

programa

: importacion* seccionVariables? seccionPrincipal EOF

importacion

: 'import' rutaImportacion ';'?

rutaImportacion

: IDENTIFICADOR ('.' IDENTIFICADOR)+

seccionVariables

: 'VARIABILES' '>' declaracion*

seccionPrincipal

: 'MAIOR' '>' instruccion* 'FINIS' ';'

declaracion

: declaracionVariable

| declaracionArreglo

declaracionVariable

: 'esto' IDENTIFICADOR ':' cuerpoDeclaracion ';'

cuerpoDeclaracion

: tipo inicializador?

| ('verum' | 'falsus')

| expresion ':' IDENTIFICADOR

| creacionObjeto

declaracionArreglo

: 'series' IDENTIFICADOR dimension+ ':' tipo inicializador? ';'

dimension

: '[' expresion ']'

inicializador

: expresion

| listaInicializacion

listaInicializacion

: '{' (inicializador (',' inicializador)*)? '}'

tipo

: 'numerus'

| 'textum'

| 'decimalis'

| 'littera'

| 'bool'

| IDENTIFICADOR

bloque

: '{' instruccion* '}'

instruccion

: declaracion

| asignacion

| incremento

| llamada

| condicional

| cicloDum

| cicloFacere

| cicloPer

| continuar

| interrumpir

| lectura

| escritura

asignacion

: acceso '=' expresion ';'

incremento

: acceso ('++' | '--') ';'

llamada

: acceso ';'

continuar

: 'perge' ';'

interrumpir

: 'interrumpe' ';'

condicional

: 'si' '(' expresion ')' bloque sinoSi* sino? 'finis' ';'

sinoSi

: 'aliter' '(' expresion ')' bloque

sino

: 'aliter' bloque

cicloDum

: 'dum' '(' expresion ')' bloque 'finis' ';'

cicloFacere

: 'facere' bloque 'dum' '(' expresion ')' ';'

cicloPer

: 'per' '(' inicioPer ';' expresion ';' actualizacionPer ')' bloque ('finis' ';')?

inicioPer

: 'esto' IDENTIFICADOR ':' tipo expresion

| acceso '=' expresion

actualizacionPer

: acceso ('++' | '--')

| acceso '=' expresion

lectura

: acceso? '<<' ';'?

escritura

: '>>' expresion ('>>' expresion)* ';'

acceso

: IDENTIFICADOR sufijo*

sufijo

: '[' expresion ']'

| '.' IDENTIFICADOR

| '(' argumentos? ')'

argumentos

: expresion (',' expresion)*

creacionObjeto

: 'novus' IDENTIFICADOR '(' argumentos? ')'

expresion

: '(' expresion ')'

| literal

| creacionObjeto sufijo*

| acceso

| ('non' | '!' | '-') expresion

| expresion ('*' | '/') expresion

| expresion ('+' | '-') expresion

| expresion ('<' | '<=' | '>' | '>=') expresion

| expresion ('==' | '!=') expresion

| expresion '&&' expresion

| expresion '||' expresion

literal

: ENTERO

| DECIMAL

| TEXTO

| CARACTER

| 'verum'

| 'falsus'

| 'null'

Y

programa

: NEWLINE* seccionEstructuras? seccionFunciones EOF

seccionEstructuras

: '%estructuras' NEWLINE definicionEstructura*

seccionFunciones

: '%funciones' NEWLINE definicionFuncion*

definicionEstructura

: 'estructura' IDENTIFICADOR ':' NEWLINE INDENT atributoEstructura+ DEDENT

atributoEstructura

: tipo IDENTIFICADOR dimension* NEWLINE

definicionFuncion

: 'definir' IDENTIFICADOR '(' parametros? ')' ('->' tipoRetorno)? ':' NEWLINE bloque

tipoRetorno

: tipo ('[' ']')*

parametros

: parametro (',' parametro)*

parametro

: tipo IDENTIFICADOR

| ('[' ']')+ tipo IDENTIFICADOR

| '{' '}' IDENTIFICADOR IDENTIFICADOR

tipo

: 'entero'

| 'flotante'

| 'cadena'

| 'caracter'

| 'bool'

| IDENTIFICADOR

bloque

: INDENT instruccion+ DEDENT

instruccion

: instruccionSimple ';'? NEWLINE

| instruccionCompuesta

instruccionSimple

: declaracion

| asignacion

| incremento

| llamadaFuncion

| imprimir

| leer

| 'romper'

| 'continuar'

| retornar

instruccionCompuesta

: definicionEstructura

| condicional

| elegir

| cicloPara

| cicloMientras

| cicloHacer

declaracion

: tipo IDENTIFICADOR dimension* ('=' inicializador)?

```
dimension
: '[' expresion ']'
inicializador
: expresion
| listaInicializacion
listaInicializacion
: '{' (inicializador (',' inicializador)*)? '}'
asignacion
: acceso '=' inicializador
incremento
: acceso ('++' | '--')
retornar
: 'retornar' expresion?
imprimir
: 'imprimir' '(' argumentos? ')'
leer
: 'leer' '(' ')'
llamadaFuncion
: IDENTIFICADOR '(' argumentos? ')'
argumentos
: expresion (',' expresion)*
condicional
: 'si' '(' expresion ')' 'entonces' NEWLINE bloque sinoSi* contrario?
sinoSi
: 'sino' '(' expresion ')' 'entonces' NEWLINE bloque
```

```
contrario
: 'contrario' NEWLINE bloque
elegir
: 'elegir' '(' expresion ')' ':' NEWLINE INDENT caso* casoSiempre? DEDENT
caso
: 'caso' expresion ':' NEWLINE bloque
casoSiempre
: 'siempre' ':' NEWLINE bloque
cicloPara
: 'para' '(' inicioPara? ';' expresion? ';' actualizacionPara? ')' ':' NEWLINE bloque
inicioPara
: declaracion
| asignacion
actualizacionPara
: incremento
| asignacion
cicloMientras
: 'mientras' '(' expresion ')' 'hacer' NEWLINE bloque
cicloHacer
: 'hacer' ':' NEWLINE bloque 'mientras' '(' expresion ')' ';'? NEWLINE
acceso
: IDENTIFICADOR sufijoAcceso*
sufijoAcceso
: '[' expresion ']'
| '.' IDENTIFICADOR
```

```
expresion
: '(' expresion ')'
| literal
| leer
| llamadaFuncion
| acceso
| ('!' | '-') expresion
| expresion ('*' | '/' | '%') expresion
| expresion ('+' | '-') expresion
| expresion ('<' | '>' | '<=' | '>=') expresion
| expresion ('==' | '!=') expresion
| expresion '&&' expresion
| expresion '||' expresion
literal
: ENTERO
| DECIMAL
| TEXTO
| CARACTER
| 'verdadero'
| 'falso'
```

ZETARIANO

```
compilacion
: declaracionClase EOF
declaracionClase
```

: modificador* 'class' IDENTIFICADOR ('extends' IDENTIFICADOR)? '{' miembro* '}'

modificador

: 'public'

| 'private'

| 'protected'

| 'static'

miembro

: declaracionAtributo

| declaracionConstructor

| declaracionMetodo

declaracionAtributo

: modificador* tipo declaradorVariable (',' declaradorVariable)* ';'

declaracionConstructor

: modificador* IDENTIFICADOR '(' parametros? ')' cuerpoConstructor

cuerpoConstructor

: '{' llamadaSuper? sentencia* '}'

llamadaSuper

: 'super' '(' argumentos? ')' ';'

declaracionMetodo

: '@Override'? modificador* tipoRetorno IDENTIFICADOR '(' parametros? ')' bloque

tipoRetorno

: tipo

| 'void'

```
parametros
: parametro (',' parametro)*
parametro
: tipo IDENTIFICADOR
tipo
: tipoBase ('[' ']')*
tipoBase
: 'int'
| 'double'
| 'char'
| 'boolean'
| 'String'
| IDENTIFICADOR
declaradorVariable
: IDENTIFICADOR ('=' inicializadorVariable)?
inicializadorVariable
: expresion
| inicializadorArreglo
inicializadorArreglo
: '{' (inicializadorVariable (',' inicializadorVariable)* ','?)? '}'
bloque
: '{' sentencia* '}'
sentencia
: bloque
| declaracionLocal ';'
```

| 'if' '(' expresion ')' sentencia ('else' sentencia)?

| 'switch' '(' expresion ')' '{' seccionSwitch* '}'

| 'for' '(' inicioFor? ';' expresion? ';' listaExpresiones? ')' sentencia

| 'while' '(' expresion ')' sentencia

| 'do' sentencia 'while' '(' expresion ')' ';'

| 'break' ';'

| 'continue' ';'

| 'return' expresion? ';'

| 'println' '(' expresion? ')' ';'

| 'print' '(' expresion ')' ';'

| expresion ';'

| ';'

declaracionLocal

: tipo declaradorVariable (',' declaradorVariable)*

seccionSwitch

: etiquetaSwitch+ sentencia*

etiquetaSwitch

: 'case' expresion ':'

| 'default' ':'

inicioFor

: declaracionLocal

| listaExpresiones

listaExpresiones

: expresion (',' expresion)*

expresion

: primario

| expresion '.' IDENTIFICADOR '(' argumentos? ')'

| expresion '.' IDENTIFICADOR

| expresion '[' expresion ']'

| expresion ('++' | '--')

| ('++' | '--' | '+' | '-' | '!') expresion

| expresion ('*' | '/' | '%') expresion

| expresion ('+' | '-') expresion

| expresion ('<' | '>' | '<=' | '>=') expresion

| expresion ('==' | '!=') expresion

| expresion '&&' expresion

| expresion '||' expresion

| <right> expresion '?' expresion ':' expresion

| <right> expresion ('=' | '+=' | '-=' | '*=' | '/=' | '%=') expresion

primario

: '(' expresion ')'

| literal

| 'this'

| 'super' '.' IDENTIFICADOR '(' argumentos? ')'

| 'super' '.' IDENTIFICADOR

| IDENTIFICADOR '(' argumentos? ')'

| IDENTIFICADOR

| 'new' IDENTIFICADOR '(' argumentos? ')'

| 'new' tipoBase ('[' expresion ']')+ ('[' ']')*

| 'new' tipoBase ('[' ']')+ inicializadorArreglo

| 'readln' '(' ')'

argumentos

: expresion (',' expresion)*

literal

: ENTERO

| DECIMAL

| TEXTO

| CARACTER

| 'true'

| 'false'

| 'null'

ANALIZADOR SEMÁNTICO

El análisis semántico se hace dentro de los visitors, directamente sobre el árbol que genera ANTLR. Cada lenguaje tiene dos visitors y todos los archivos del programa pasan por ellos en dos recorridos:

1. Declaraciones: registran en las tablas las estructuras, clases, atributos, funciones, métodos y constructores, y cargan los archivos que importa el .pig.
2. Traducción: validan cada declaración, sentencia y expresión y, en el mismo recorrido, generan sus cuartetas. Este recorrido empieza cuando todos los archivos registran sus declaraciones, así un archivo puede usar una clase o una función definida en otro. Si hay algún error no se entregan cuartetas.

Validaciones que se aplican:

Regla	Cómo se valida
Imports válidos	El archivo debe existir, tener extensión .y o .z y no estar repetido.
Nombre de clase = nombre de archivo	La clase de un .z debe llamarse igual que el archivo.
Nombres repetidos	Variables en el mismo ámbito, atributos, funciones, métodos y constructores con la misma firma, y estructuras o clases con el mismo nombre.
Tipos definidos	Todo tipo usado debe existir en la tabla de tipos, aunque esté definido en otro archivo.
Declarado antes de usarse	La variable se busca desde el ámbito actual hacia afuera sin salir de la función, y su declaración debe estar antes del uso.
Compatibilidad de tipos	Asignaciones, valores iniciales, operaciones, argumentos y retornos se validan con la

	tabla de compatibilidad; un entero se promueve a decimal en cualquier expresión.
Condiciones	si, dum, per, if, while, for, do, mientras y el ternario deben tener una condición booleana.
Arreglos	Los índices y tamaños deben ser enteros; solo se indexa lo que es arreglo.
Listas {}	Inicializan arreglos o estructuras.
Llamadas	La función, método o constructor debe existir; se elige la sobrecarga cuyos parámetros aceptan los argumentos.
Objetos y null	novus / new solo de clases y los atributos y métodos deben existir; null se puede asignar a cualquier objeto o arreglo.
Herencia	Solo herencia simple: la clase padre debe existir y no puede haber ciclos de herencia.
Visibilidad	private y sin modificador: solo dentro de la clase; protected: la clase y sus subclases; public: desde cualquier lugar.
@Override	Obligatorio al sobrescribir un método y error si no sobrescribe ninguno; se permite un retorno covariante y no se puede reducir la visibilidad.
super	super(...) solo como primera instrucción de un constructor; si no se escribe, se llama al constructor vacío del padre y es error si no existe. También super.metodo() y super.atributo.
Retorno	return con valor en funciones con tipo y sin valor en las que no tienen tipo.

break/continue	perge, interrumpe, romper, continuar, break y continuar solo dentro de ciclos; romper y break también dentro de elegir y switch.
Asignación en Zetariano	Solo a una variable, un atributo o una posición de arreglo.

CÓDIGO DE TRES DIRECCIONES

La traducción se hace en el mismo recorrido que el análisis semántico: cada sentencia válida genera sus cuartetas y de ellas se obtiene el texto del código de tres direcciones, todo se expresa con temporales, etiquetas, saltos y accesos al stack y al heap.

- **Memoria:** el stack y el heap son arreglos de celdas. P apunta al inicio del marco de la función actual y H a la siguiente celda libre del heap; main empieza con P = 0 y H = 1, y null es 0.
- **Marco de una función:** stack[P + 0] guarda el valor de retorno, desde stack[P + 1] van this y los parámetros, y después las variables locales y los temporales.
- **Llamadas:** el llamador escribe los argumentos en el marco siguiente, suma a P el tamaño de su marco, ejecuta call, resta el tamaño y lee el valor de retorno.
- **Cadenas:** se guardan en el heap como [largo, c1, …, cn]. La concatenación, la comparación y las conversiones entre texto y números se generan en línea con ciclos.
- **Objetos y arreglos:** viven en el heap. La celda 0 de cada objeto guarda el número de su clase, y el despacho dinámico compara ese número para saltar a la implementación del método que corresponde.
- **Entrada y salida:** print imprime enteros, decimales y caracteres, y leer_caracter lee un carácter; leer una línea es un ciclo que guarda los caracteres en el heap.

TRADUCCIÓN A RISC-V

La clase TraductorRiscV recorre las cuartetas y genera ensamblador RV32IMFD para el simulador RARS. Al iniciar, el programa pide la memoria del stack y del heap con la llamada al sistema sbrk. Cada celda ocupa 8 bytes: un entero usa la palabra baja (lw/sw) y un decimal la celda completa (fld/fsd). Cada temporal vive en una celda de su marco, por lo que la recursión no los

pisa, y su tipo (entero o decimal) se deduce de las cuartetas que lo asignan. Los registros se usan así:

- **s0 y s1:** dirección del stack y del heap.
- **s2 y s3:** P y H, como número de celda.
- **s4:** dirección de stack[P], el frame pointer en bytes; se recalcula cada vez que cambia P.
- **t0, t1, t2 y ft0 a ft3:** operandos y resultados de cada instrucción; t5 y t6 calculan direcciones.
- **ra y sp:** la dirección de retorno se guarda en la pila del procesador al entrar a cada función.
- **a0, a7 y fa0:** llamadas al sistema de RARS (1 imprimir entero, 3 imprimir decimal, 9 sbrk, 10 terminar, 11 imprimir carácter y 12 leer carácter).

Un salto condicional de RISC-V solo llega a 4 KB, así que cada if del código de tres direcciones se traduce como la condición contraria sobre un j, que llega a 1 MB. Sobre cada grupo de instrucciones se deja como comentario la instrucción de tres direcciones que traduce.

TABLA DE COLOREADO

Categoría	Color
Control de flujo (si, dum, per, if, while, switch, mientras, elegir, retornar…)	#8E24AA
Declaraciones y modificadores (import, esto, series, novus, class, public, extends, new, definir, estructura)	#283593
Tipos de dato (numerus, decimalis, int, double, String, entero, flotante, void…)	#E65100
Valores especiales (verum, falsus, true, false, verdadero, falso, null, this, super)	#AD1457
Anotación @Override	#9E7C00
Entrada y salida (>>, <<, print, println, readln, imprimir, leer)	#7B1E3A
Secciones (VARIABILES>, MAIOR>, FINIS, %estructuras, %funciones)	#00ACC1
Nombres de clases y estructuras	#00695C
Funciones y métodos	#1565C0
Números	#C62828
Cadenas y caracteres	#388E3C
Comentarios	#8A8A8A
Operadores	#546E7A

Signos de agrupación ( ) [ ] { }	#455A64
Puntuación ; , . :	#616161
Identificadores	#1E1E1E
Errores léxicos	#D50000

INTERFAZ GRÁFICA

Se propuso este diseño amigable con el usuario.

